NAME : HAMZA

ROLL_NO: 25i-0686

SECTION : F

REPORT FOR THE PHASE OF DSA ASSIGNMENT 2

Stack and Queue Representation:

The implementation which i chose for the stack was the linked list based implementation and below are the reasons:

1. It would have any specific size, it can grow as much as it wants .
2. You wouldn't have to care about the indexes and most of your focus is going to be on the task at front.
3. Less validation checks.
4. After using the linked list implementation multiple times in the quizzes and lab tasks I have become way more comfortable with it rather than the array based implementation.
5. I wouldn't have to care about the out of bounds access.

The implementation which i chooose for the circular queue was the array based implementation and thats because it was explicitly mentioned on the startr code in the section 5 that is the required class interfaces area that the queueType must be circular queue that is why i chose this type of implementation of the queue.

EACH OPERATION COMPLEXITY

Stack:

StackType() : The time complexity is O(1) because it only sets the head and the counters

Push : O(1) Adds one node at the head

Pop: O(1) Removes the head node

Top Reads the head node O(1)

Size, isEmptyStack, isFullStack : O(1) reads count or the head pointer

Intiailze stack ; deletes every node O(n)

Destructor : deletes every node O(n)

Copy Constructor : O(n) copies every node

PeekAt(i) : O(n) walks every node and in worst case till the end

Circular queue :

Queuetype(size) : O(capacity) allocates the array

AddQueue: O(1) one write at rear index

DeleteQueue : O(1) only moves front index

Front, back : O(1) direct array access

Size , capacity, isEmptyQueue, isFullQueue : O(1) Compare or read count and the capacity

InitializeQueue : O(1) resets the indices and the count onlly

Destructor : O(capacity) frees the array

Copy Constructor : O(capacity) allocates the array

PeekAt(i) : O(1) direct index with

QUEUE Policy :

My queeue policy is that when the whole queue is filled and one at the front gets deleted according to the FIFO memory philosophy then since it is circular then the other one being added is going to be behing the front and you can say that in that case the rear is going to be behing the front.

Test results:

All operations and components were tested and all tests were passed.